

PRACTICAL : WEEK 8

Task -1:

Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

CODE:

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *ptr1;
    int *ptr2;

    // malloc()
    ptr1 = (int *)malloc(5 * sizeof(int));

    if (ptr1 == NULL)
    {
        printf("malloc failed.\n");
        return 1;
    }

    printf("Memory allocated using malloc().\n");

    for (int i = 0; i < 5; i++)
    {
        ptr1[i] = i + 1;
    }

    // calloc()
    ptr2 = (int *)calloc(5, sizeof(int));

    if (ptr2 == NULL)
    {
        printf("calloc failed.\n");
        free(ptr1);
        return 1;
    }

    printf("Memory allocated using calloc().\n");

    // realloc()
    ptr1 = (int *)realloc(ptr1, 10 * sizeof(int));

    if (ptr1 == NULL)
    {
        printf("realloc failed.\n");
        free(ptr2);
        return 1;
    }
}
```

```
printf("Memory reallocated using realloc().\n");

for (int i = 5; i < 10; i++)
{
    ptr1[i] = i + 1;
}

printf("Values after realloc():\n");

for (int i = 0; i < 10; i++)
{
    printf("%d ", ptr1[i]);
}

printf("\n");

// free()
free(ptr1);
free(ptr2);

printf("Memory released using free().\n");

return 0;
}
```

OUTPUT:

```
ananya@Ananya:~$ cd /mnt/d/OSSP_Programs/Practical/Week8
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week8$ nano memory_alloc.c
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week8$ gcc memory_alloc.c -o memory_alloc
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week8$ ./memory_alloc
Memory allocated using malloc().
Memory allocated using calloc().
Memory reallocated using realloc().
Values after realloc():
1 2 3 4 5 6 7 8 9 10
Memory released using free().
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week8$ |
```

OBSERVATION:

The program successfully demonstrated dynamic memory allocation using `malloc()`, `calloc()`, and `realloc()`, and released the allocated memory using `free()`. The `malloc()` function allocated memory without initialization, `calloc()` allocated and initialized memory to zero, and `realloc()` successfully increased the allocated memory size. Valgrind was used to monitor the program's memory usage and identify memory leaks. After properly using `free()`, Valgrind confirmed that all allocated memory was released and no memory leaks were present. Thus, dynamic memory allocation and memory leak detection using Valgrind were successfully demonstrated.

Task -2:

Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process. CODE:

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define SIZE 1000000

int main()
{
    int *data;

    data = malloc(SIZE * sizeof(int));

    if (data == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    for (int i = 0; i < SIZE; i++)
    {
        data[i] = 10;
    }

    printf("Parent PID: %d\n", getpid());
    printf("Memory allocated for %d integers.\n", SIZE);
    printf("Address of data: %p\n", (void *)data);

    pid_t pid = fork();

    if (pid < 0)
    {
        printf("Fork failed.\n");
        free(data);
        return 1;
    }

    if (pid == 0)
    {
        printf("\nChild process created.\n");
        printf("Child PID: %d\n", getpid());

        printf("Before modification: data[0] = %d\n", data[0]);

        sleep(2);

        data[0] = 999;

        printf("After modification: data[0] = %d\n", data[0]);
        printf("Child address of data: %p\n", (void *)data);

        sleep(5);

        free(data);
        return 0;
    }
    else
    {
        printf("\nParent is waiting for child...\n");

        sleep(4);

        printf("Parent data[0] = %d\n", data[0]);

        wait(NULL);

        free(data);

        printf("Parent process finished.\n");
    }

    return 0;
}
```

OUTPUT:

```
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week8$ nano cow.c
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week8$ gcc cow.c -o cow
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week8$ ./cow
Parent PID: 648
Memory allocated for 1000000 integers.
Address of data: 0x78ee69e2f010

Parent is waiting for child...

Child process created.
Child PID: 649
Before modification: data[0] = 10
After modification: data[0] = 999
Child address of data: 0x78ee69e2f010
Parent data[0] = 10
Parent process finished.
```

OBSERVATION:

The program successfully demonstrated Copy-on-Write (COW) behavior after using `fork()`. Initially, the parent and child processes shared the same memory pages. When the child modified the value of `data[0]`, the operating system created a separate copy of the affected memory page for the child. As a result, the child displayed the modified value while the parent retained the original value. The memory usage of the parent and child processes was analyzed using `pmap` and `/proc/<PID>/maps`. Thus, the Copy-on-Write mechanism and its effect on memory usage after `fork()` were successfully demonstrated.